

Lesson 12

■ Architectural Design

Slide Set to accompany

Software Engineering: A Practitioner's Approach, 8/e

by Roger S. Pressman and Bruce R. Maxim

Slides copyright © 1996, 2001, 2005, 2009, 2014 by Roger S. Pressman

For non-profit educational use only

May be reproduced ONLY for student use at the university level when used in conjunction with *Software Engineering: A Practitioner's Approach, 8/e*. Any other reproduction or use is prohibited without the express written permission of the author.

All copyright information MUST appear if these slides are posted on a website for student use.

Software Design

Design is an activity concerned with making major decisions, often of a structural nature.

It shares with programming a concern for abstracting information representation and processing sequences, but the level of detail is quite different at the extremes.

Design builds coherent, well-planned representations of programs that concentrate on the interrelationships of parts at the higher level and the logical operations involved at the lower levels.

What is Architecture?

The **software architecture** of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.

Why Architecture?

The architecture is not the operational software. Rather, it is a representation that enables a software engineer to:

- (1) analyze the **effectiveness of the design** in meeting its stated requirements.
- (2) consider **architectural alternatives** at a stage when making design changes is still relatively easy.
- (3) **reduce the risks** associated with the construction of the software.

Why is Architecture Important?

- **Representations of software architecture are an enabler** for communication between all parties (stakeholders) interested in the development of a computer-based system.
- **The architecture highlights early design decisions** that will have a profound impact on all software engineering work that follows and, as important, on the ultimate success of the system as an operational entity.
- **Architecture “constitutes a relatively small, intellectually graspable mode** of how the system is structured and how its components work together” [BAS03].

Formal Descriptions of Architectural

- The IEEE Computer Society has proposed IEEE-Std-1471-2000, *Recommended Practice for Architectural Description of Software-Intensive System*, [IEE00]
 - to establish a conceptual framework and vocabulary for use during the design of software architecture,
 - to provide detailed guidelines for representing an architectural description, and
 - to encourage sound architectural design practices.

Formal Descriptions of Architectural

- The IEEE Standard defines *an architectural description* (AD) as a “a collection of products to document an architecture.”

The description itself is represented using multiple views, where each *view* is “a representation of a whole system from the perspective of a related set of [stakeholder] concerns.”

Architectural Decisions

- Each **view** developed as part of an architectural description addresses a specific stakeholder concern.
- To develop each view and the architectural description as a whole, the system architect considers a variety of alternatives and ultimately decides on the specific architectural features that best meet the concern.
- Architectural decisions themselves can be considered to be one view of the architecture. The reasons that decisions were made provide insight into the structure of a system and its conformance to stakeholder concerns.

Architectural Decisions



Architecture Decision Description Template

Each major architectural decision can be documented for later review by stakeholders who want to understand the architecture description that has been proposed. The template presented in this sidebar is an adapted and abbreviated version of a template proposed by Tyree and Ackerman [Tyr05].

Design issue:	Describe the architectural design issues that are to be addressed.
Resolution:	State the approach you've chosen to address the design issue.
Category:	Specify the design category that the issue and resolution address (e.g., data design, content structure, component structure, integration, presentation).
Assumptions:	Indicate any assumptions that helped shape the decision.
Constraints:	Specify any environmental constraints that helped shape the decision (e.g., technology standards, available patterns, project-related issues).

Alternatives:

Briefly describe the architectural design alternatives that were considered and why they were rejected.

Argument:

State why you chose the resolution over other alternatives.

Implications:

Indicate the design consequences of making the decision. How will the resolution affect other architectural design issues? Will the resolution constrain the design in any way?

Related decisions:

What other documented decisions are related to this decision?

Related concerns:

What other requirements are related to this decision?

Work products:

Indicate where this decision will be reflected in the architecture description.

Notes:

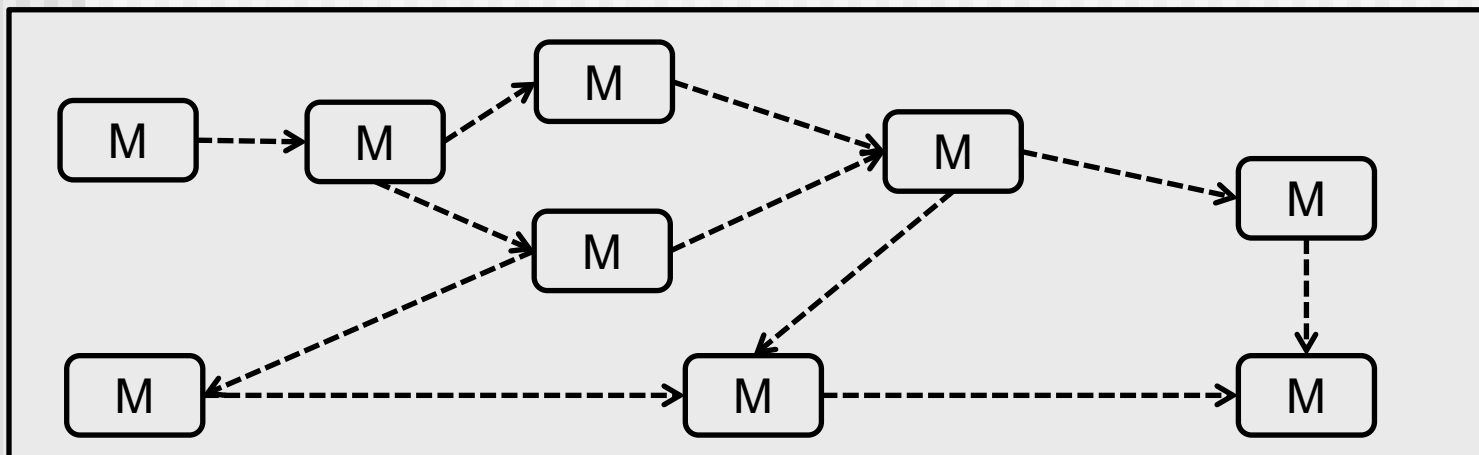
Reference any team notes or other documentation that was used to make the decision.

Agility and Architecture

- To avoid rework, user stories are used to create and evolve an architectural model (walking skeleton) before coding.
- Hybrid models which allow software architects contributing users stories to the evolving storyboard.
- Well run agile projects include delivery of work products during each sprint.
- Reviewing code emerging from the sprint can be a useful form of architectural review.

Software Architecture

软件体系结构也称软件架构，从**高层抽象**角度刻画组成软件系统的**设计元素**及它们之间的**逻辑关联**



软件体系结构如何体现了高层和抽象的特征？
这里的设计元素是什么？

Design elements of architecture

□ 构件(Component)

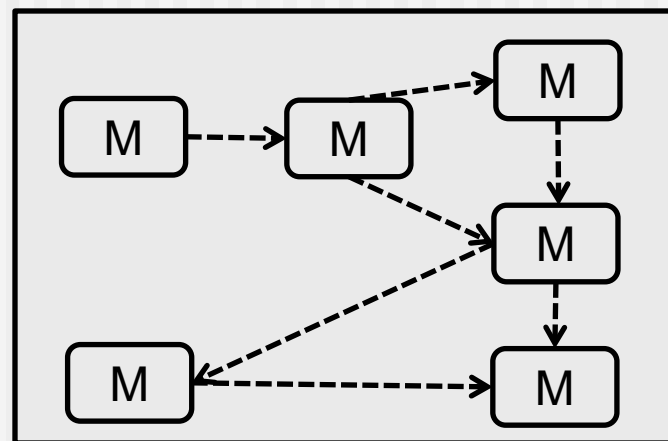
✓ 构成体系结构的基本功能部件

□ 连接件(Connector)

✓ 组件之间的连接和交互关系

□ 约束(Constraint)

✓ 组件中的元素应满足的条件以及组件经由连接件组装成更大模块时应满足的条件



Component

❑ 何为构件

- ✓ 软件系统中的**物理模块**，具有特定的功能和精确定义的对对外接口，外界可通过接口来访问它

❑ 特点

- ✓ **可分离**：一个或数个可独立部署执行码文件
- ✓ **可替换**：构件实例可被其它任何实现了相同接口的另一构件实例所替换
- ✓ **可配置**：可通过配置机制修改构件配置数据，影响构件对外服务的功能或行为
- ✓ **可复用**：构件可不经源代码修改，无需重新编译，即可应用于多个软件项目或软件产品

Example: Component

❑ .dll文件

ado	2019/8/4 17:43	文件夹	
en-GB	2019/8/4 17:38	文件夹	
en-US	2019/3/19 19:41	文件夹	
msadc	2019/3/19 19:41	文件夹	
Ole DB	2019/8/4 17:50	文件夹	
zh-CN	2019/3/19 19:41	文件夹	
wab32.dll	2019/3/19 12:45	应用程序扩展	853 KB
wab32res.dll	2019/3/19 12:45	应用程序扩展	942 KB

❑ .jar文件

- 构件不是源程序，而是可运行的二进制代码
- 构件是客观物理存在的（即有实际的文件），而非是逻辑存在的
- 构件是可访问的，以获取其功能和服务
- 构件应该是粗粒度的，而非细粒度的

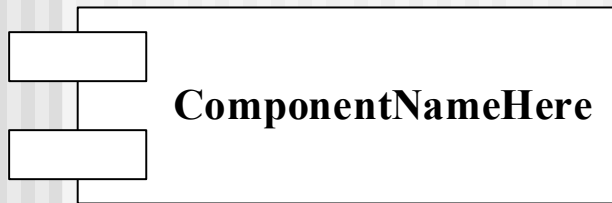
Example: Component

Acrobat软件中的软件构件

 FileInfo	2020/4/4 10:28	文件夹
 HowTo	2020/4/4 10:27	文件夹
 Javascripts	2020/4/4 10:28	文件夹
 PalmPilot	2020/4/4 10:28	文件夹
 plug_ins	2020/4/4 10:28	文件夹
 Sequences	2020/4/4 10:28	文件夹
 SPPlugins	2020/4/4 10:28	文件夹
 Updater	2020/4/4 10:28	文件夹
 WebAccess	2020/4/4 10:28	文件夹
 Ace.dll	2003/5/14 23:45	应用程序扩展
 <u>Acrobat.exe</u>	2003/5/19 12:17	应用程序
 acrobat.tlb	2003/3/27 17:47	TLB 文件
 <u>Acrofx32.dll</u>	2003/5/15 0:47	应用程序扩展
 <u>AcroIEFavClient.dll</u>	2003/5/15 1:03	应用程序扩展
 <u>AdobeUpdateManager.exe</u>	2003/5/15 0:56	应用程序
 <u>Aqm.dll</u>	2003/5/14 23:45	应用程序扩展
 <u>atl.dll</u>	2000/7/14 18:18	应用程序扩展
 AXEParse.dll	2003/5/14 23:45	应用程序扩展
 Bib.dll	2003/5/14 23:45	应用程序扩展
 CoolType.dll	2003/5/14 23:45	应用程序扩展
 cos2xpdf.xml	2003/5/15 0:01	XML 文档

UML representation of components

可以用不同的UML图符来表示软构件



Thinking and Discussion

- 举例和示范你计算机中安装的软件及其构件
- 对比软件构件与源代码、软件构件与.class之间的区别和联系
- 为什么要将软件组织为一组构件

Linker (连接子)

❑ 连接子表示软构件之间的**连接和交互**关系

- ✓ 每个软构件并非孤立，它们之间通过连接进行交互
- ✓ 交互的目的是为了交换数据、获得服务

❑ 软构件之间的典型交互方式

- ✓ 过程调用
- ✓ 远程过程调用 (Remote Procedure Call, RPC)
- ✓ 消息传递
- ✓ 事件通知和广播
- ✓ 主题订阅

Connection and Interface

❑ 二种接口(Interface)

✓ **供给接口**：对外提供的接口

✓ **需求接口**：请求其他构件帮助所需的接口

- 构件通过接口对外提供服务
- 构件通过接口与其他构件进行交互

❑ 连接端口(Port)

✓ 每个**端口**绑定了一组供给接口和/或需求接口

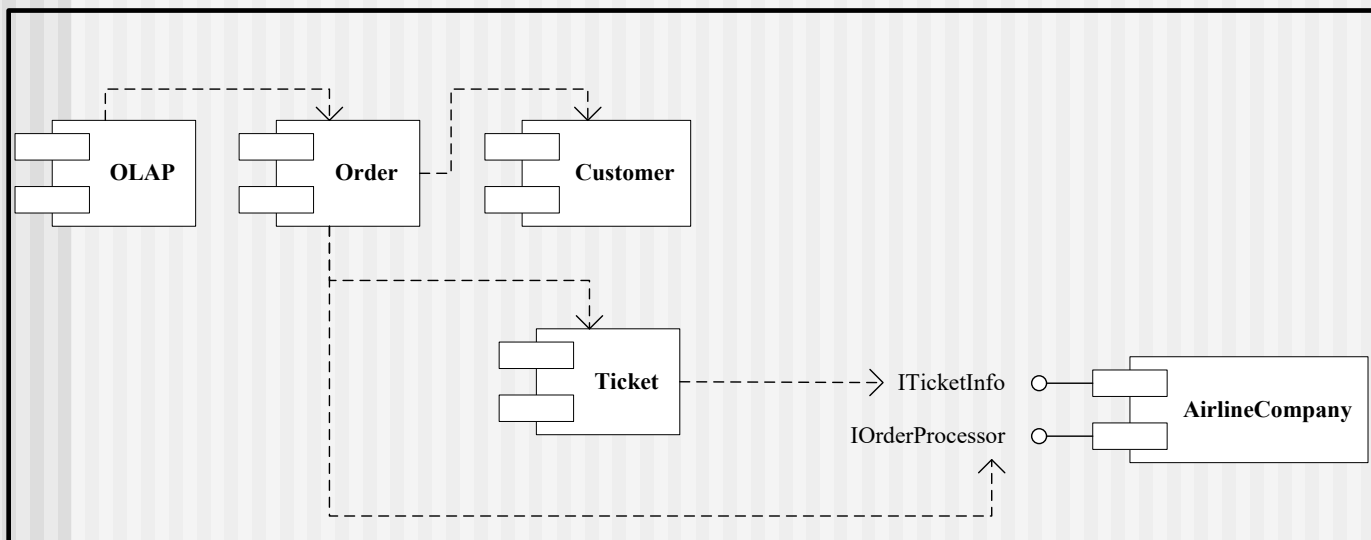
✓ 构件通过端口与外部世界交互

✓ 当外部请求到达端口时，构件的端口知道如何将外部请求路由至合适的接口的**实现体**

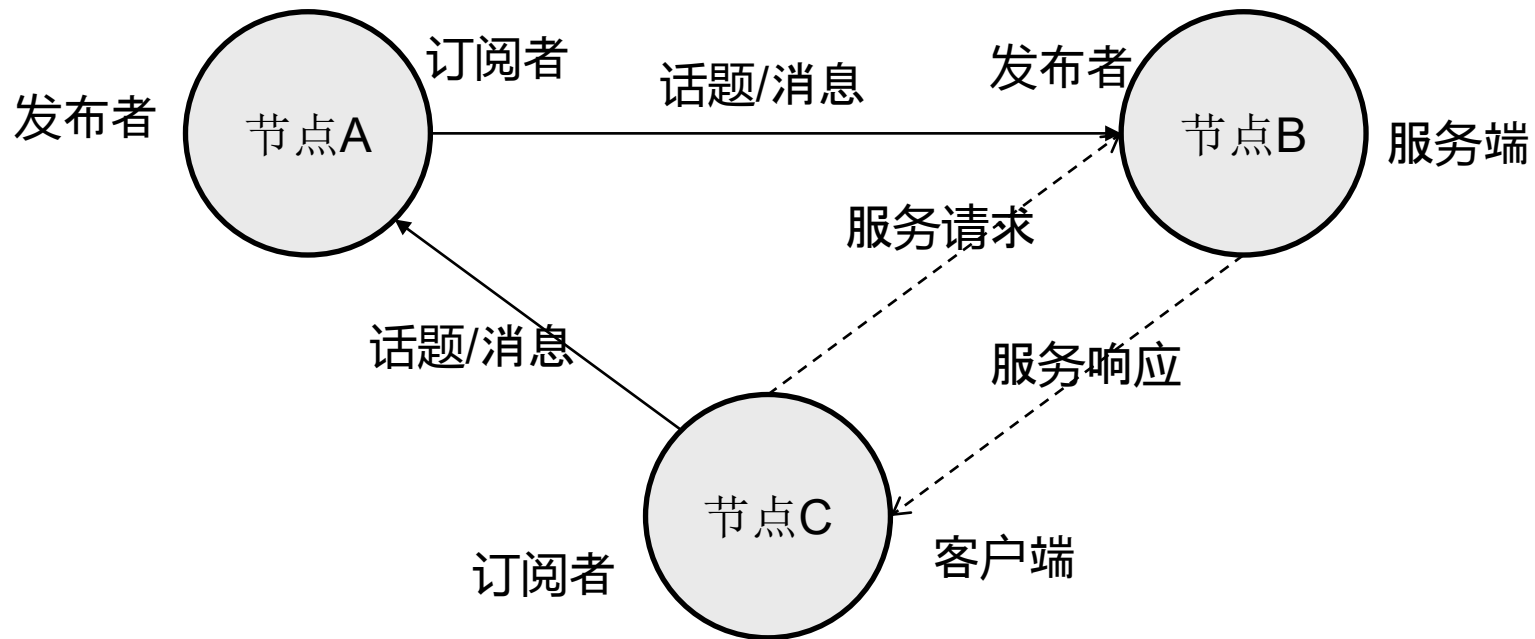
✓ 当构件通过端口请求外部服务时，端口也知道如何分辨该请求所对应的需求接口

Components and interfaces

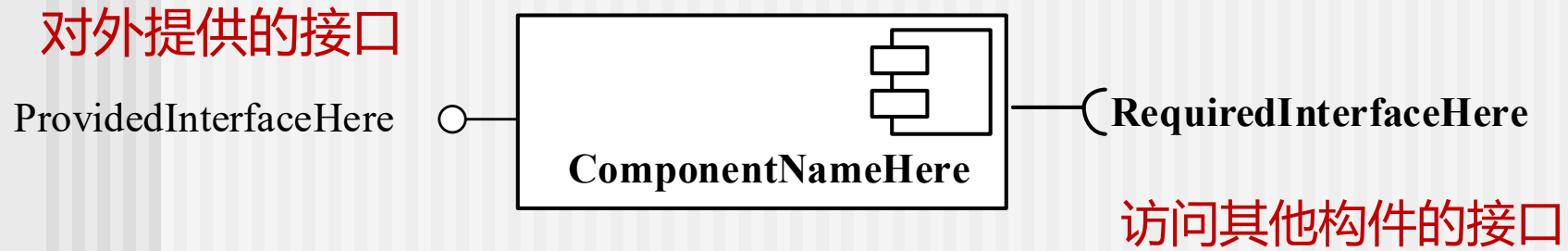
- ❑ 构件的实现与构件的接口相分离
- ❑ 构件开发者可自由选择实现方法，只要它实现供给接口中的操作及属性
- ❑ 两个构件的实现遵循相同接口定义，它们就可自由替换



Example: Connections and interactions between components



Representation of connections and interfaces



Thinking and Discussion

- ❑ 如果一个软构件没有对外的接口是否可以？
- ❑ 没有接口的软构件会带来什么问题？

Constraint

- ❑ 高层次软件元素可向低层次软件元素发请求，低层次软件元素完成计算后向高层次发送应答，反之不行
- ❑ 每个软件元素根据其职责位于适当的层次，不可错置，如核心层不能包含界面输入接收职责
- ❑ 每个层次都是可替换的，一个层次可以被实现了同样的对外服务接口的层次所替代

Different **Views** of software architecture

- ❑ **逻辑视图**：要素及关系，站在结构视点和抽象层次
- ❑ **运行视图**：在特定时刻软构件的具体情况
- ❑ **开发视图**：构件的代码组织及其形式
- ❑ **物理视图**：软构件的部署及其连接

软件体系结构是从结构的角度来刻画软件
可从不同角度、层次来表示和分析软件体系结构

Logical View of Software Architecture: Package Diagram

□ 包模型

- ✓ 刻画包间的构成和依赖关系
- ✓ 包可以包含一组具有逻辑关联的UML模型元素
- ✓ 包间关系：构成关系、依赖关系

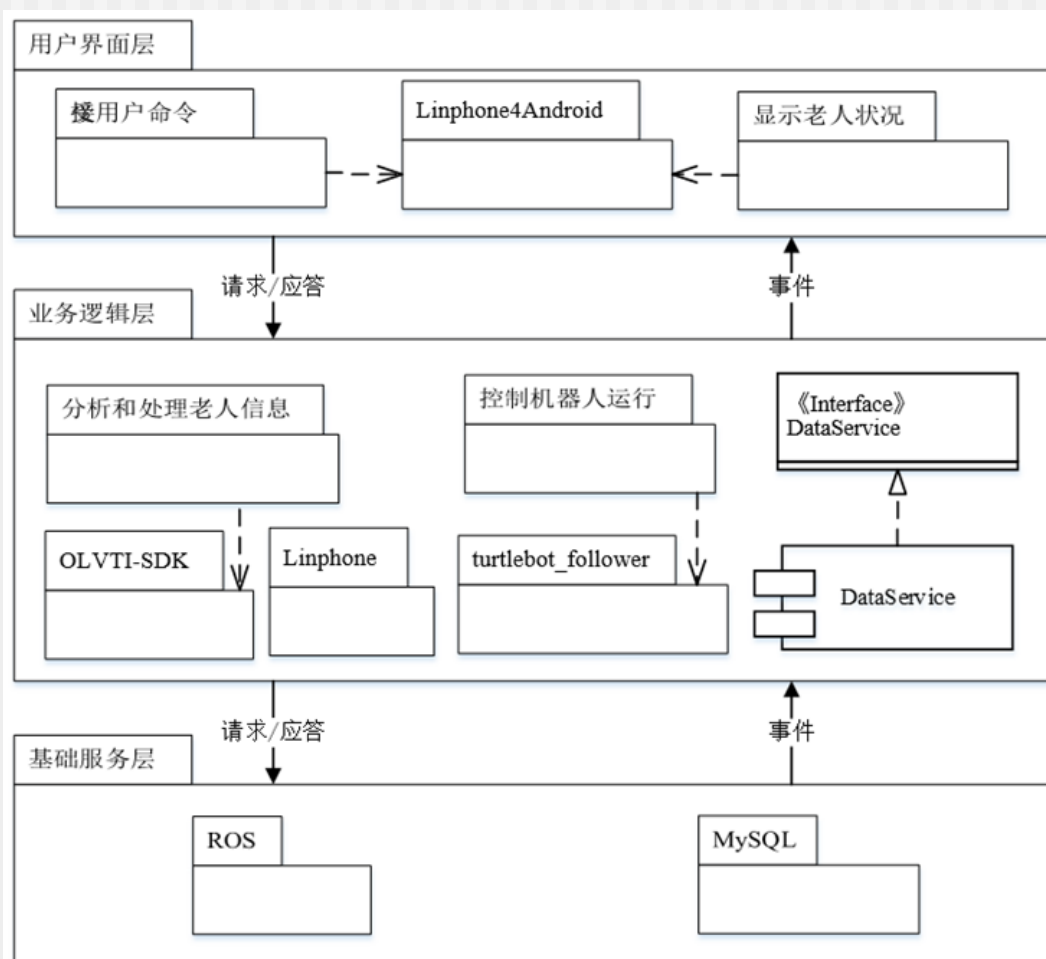
□ 包图可描述软件系统的高层结构

- ✓ 包图以结构化、层次化方式组织、管理大型的软件模型，使得分别处理不同包的开发团队之间的相互干扰程度降至最低

包图：系统（子系统）、构件及关系

Example: Package Diagram

包图表示软件体系结构



Logical View of Software Architecture: Component Diagram

❑ 描述软件系统中构件及构件间的关系

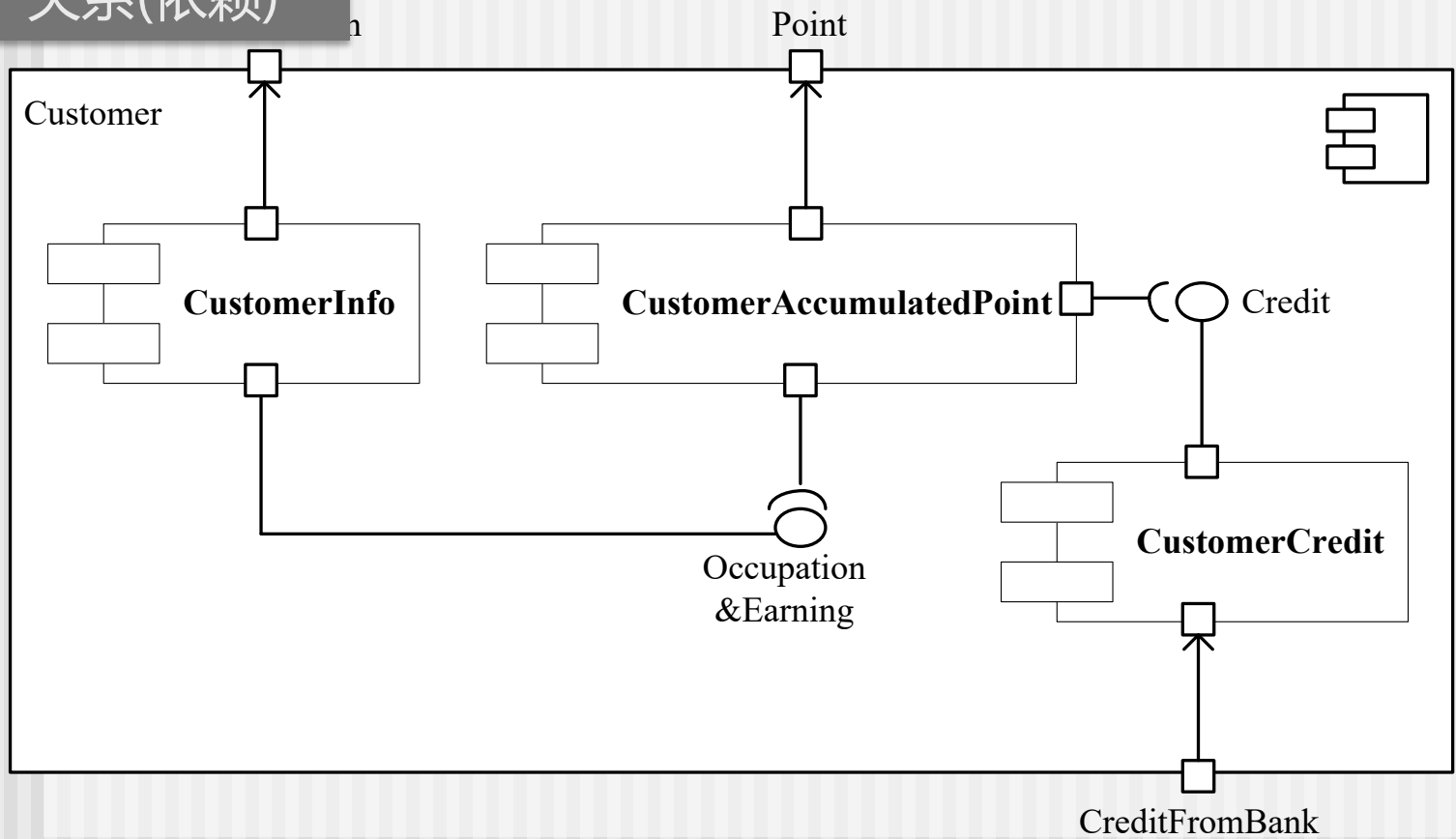
- ✓ 结点：构件、类和包
- ✓ 关系：构成关系、依赖关系

❑ 对软件体系结构的描述

- ✓ 描述软件系统中构件的接口定义及构件间的依赖关系，以便评估软件变更的影响范围
- ✓ 描述软件系统或其中某个局部的构件化设计，为在后续开发阶段实现构件化的软件模块订立设计规约

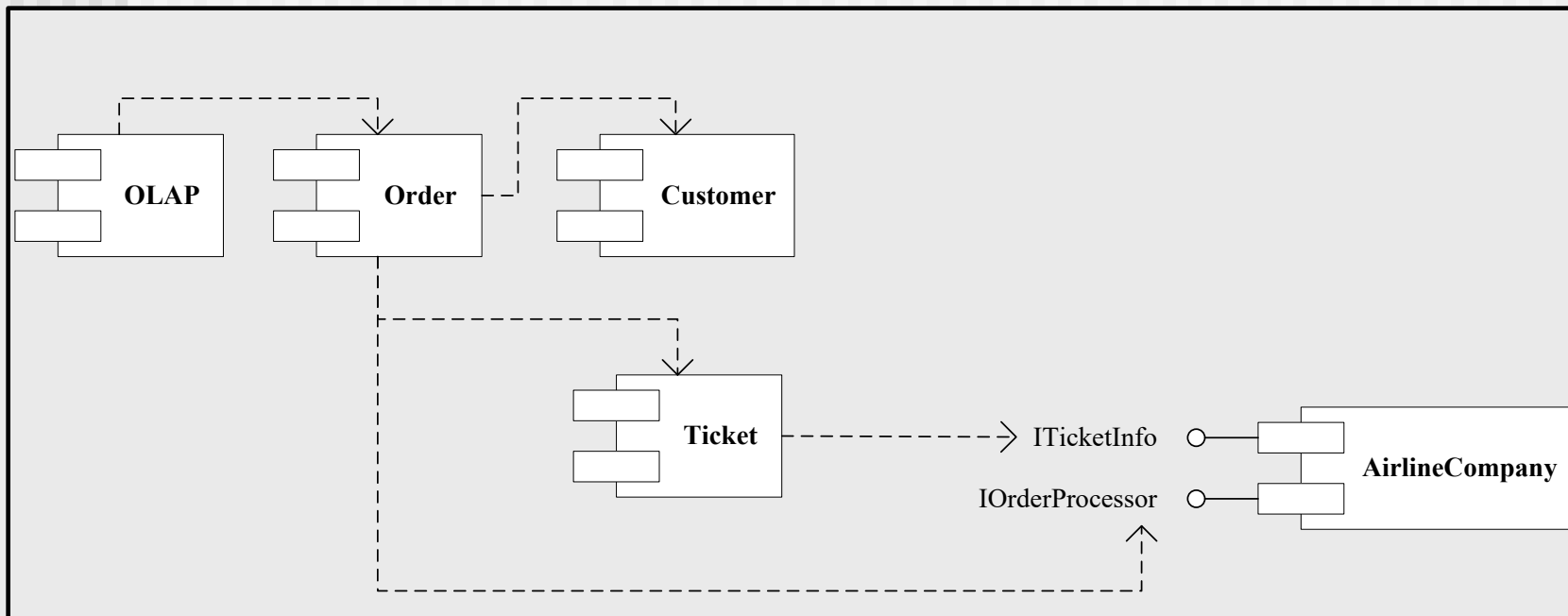
Example: Component Diagram

- 构件
- 接口和端口
- 关系(依赖)



Example: Component Diagram

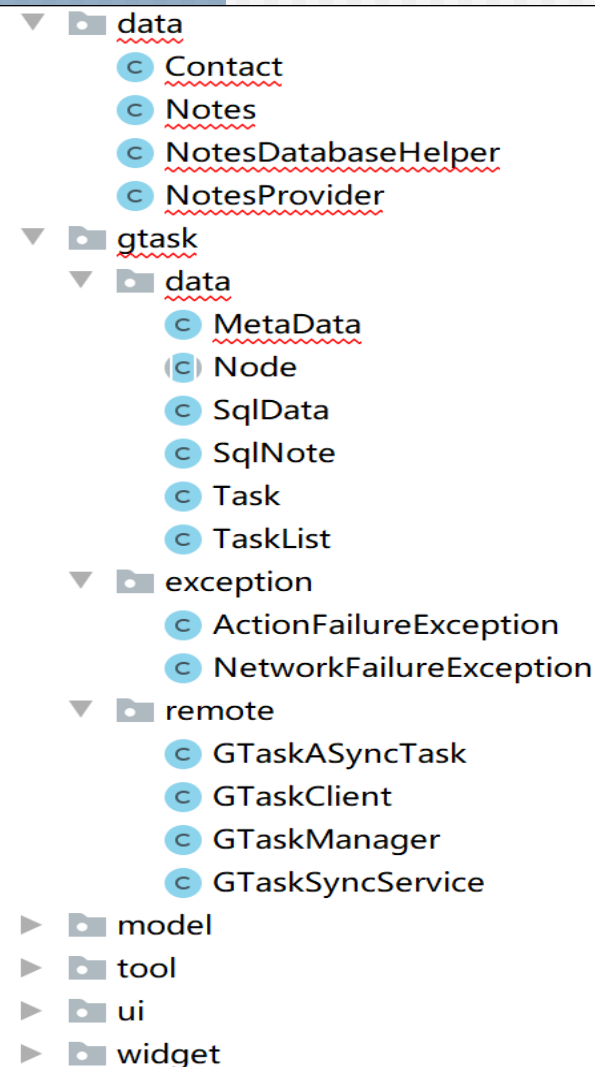
- 构件
- 接口和端口
- 关系(依赖)



Development View of Software Architecture

- ❑ 各软件要素源代码的程序分包及目录结构
- ❑ 采用的类库、中间件和框架，它们与逻辑视图中各个软件元素之间的映射关系

小米便签的代码开发视图



Deployment View of Software Architecture

❑ 部署图

- ✓ 软件安装部署的**物理机器**及其**连接**，各个件元素在这些机器上的部署位置
- ✓ 表示软件系统可执行**工件**在运行环境中的分布情况

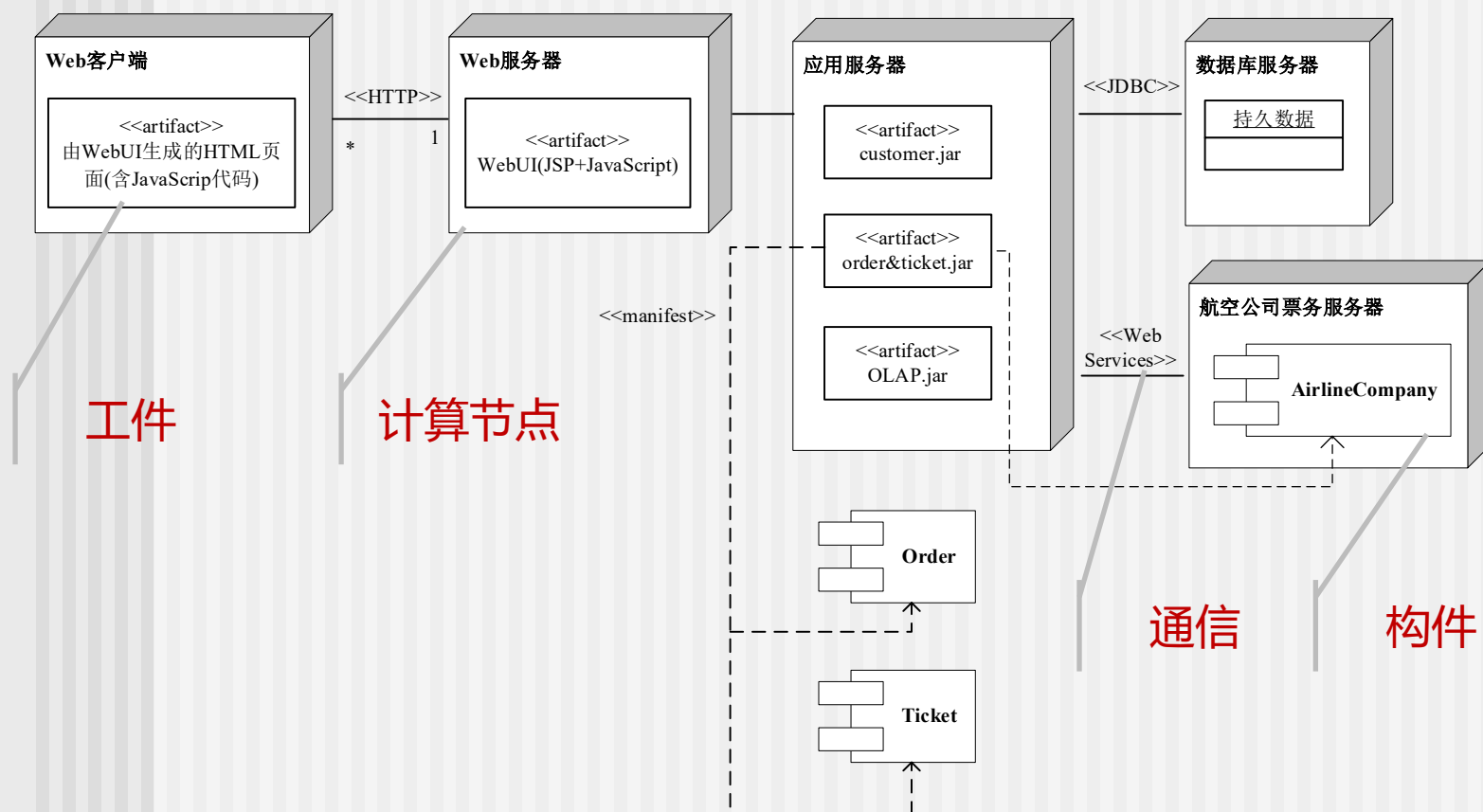
❑ 工件(Artifact)

- ✓ **工件**：软件系统中相对独立的物理实现单元
- ✓ **示例**：Windows平台下DLL文件、Java系统中类库（jar）文件

❑ 两种部署图

- ✓ 逻辑层面的描述性部署图
- ✓ 物理层面的实例性部署图

Example: Deployment View



Operational View of Software Architecture

- ❑ 软件运行时进程、线程的划分，它们之间的并发和同步，它们与逻辑视图和开发视图之间的映射关系
- ❑ 可用UML的活动图、对象图来表示

Three types of UML diagrams of software architecture

- ❑ 包图
- ❑ 构件图
- ❑ 部署图

Package Diagram

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

Package Diagram

❑ 功效

✓ 刻画包间的**构成和依赖**关系

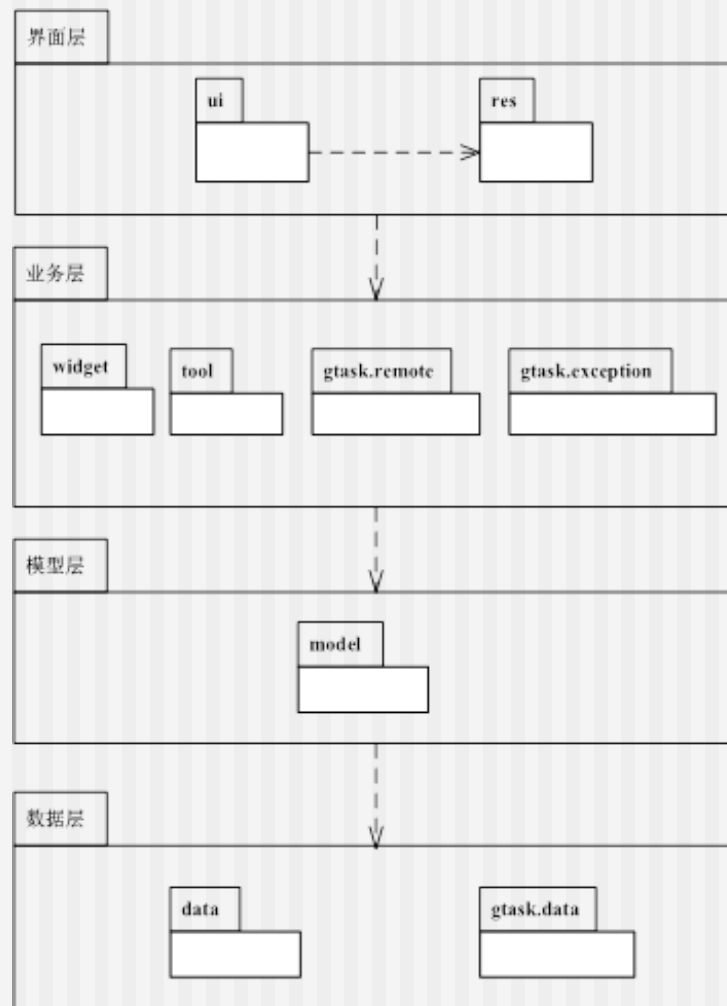
❑ 图的构成

✓ **节点**：包

✓ **边**：包间的关系

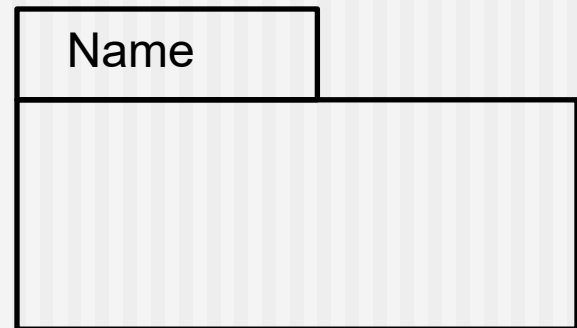
❑ 包间的关系

✓ 组成和依赖



What is a package?

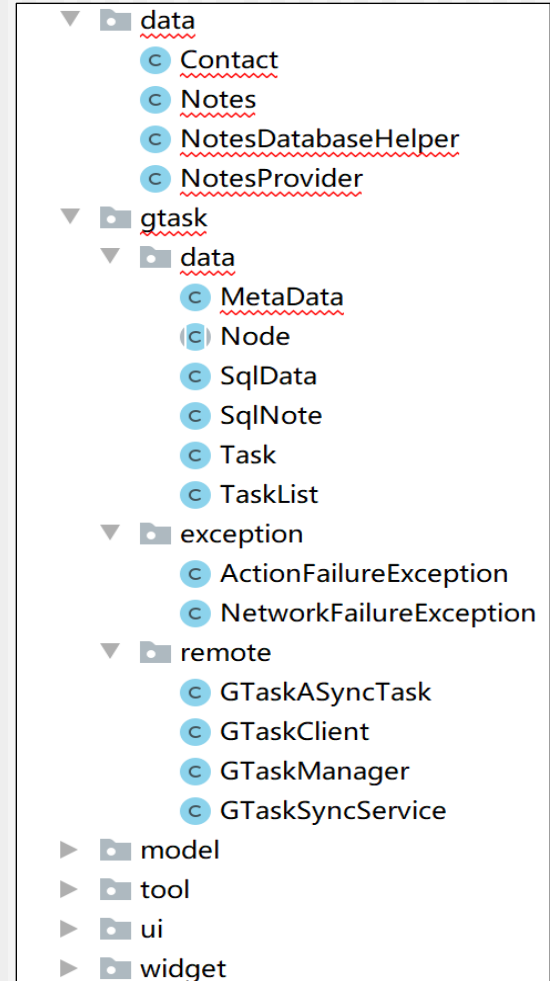
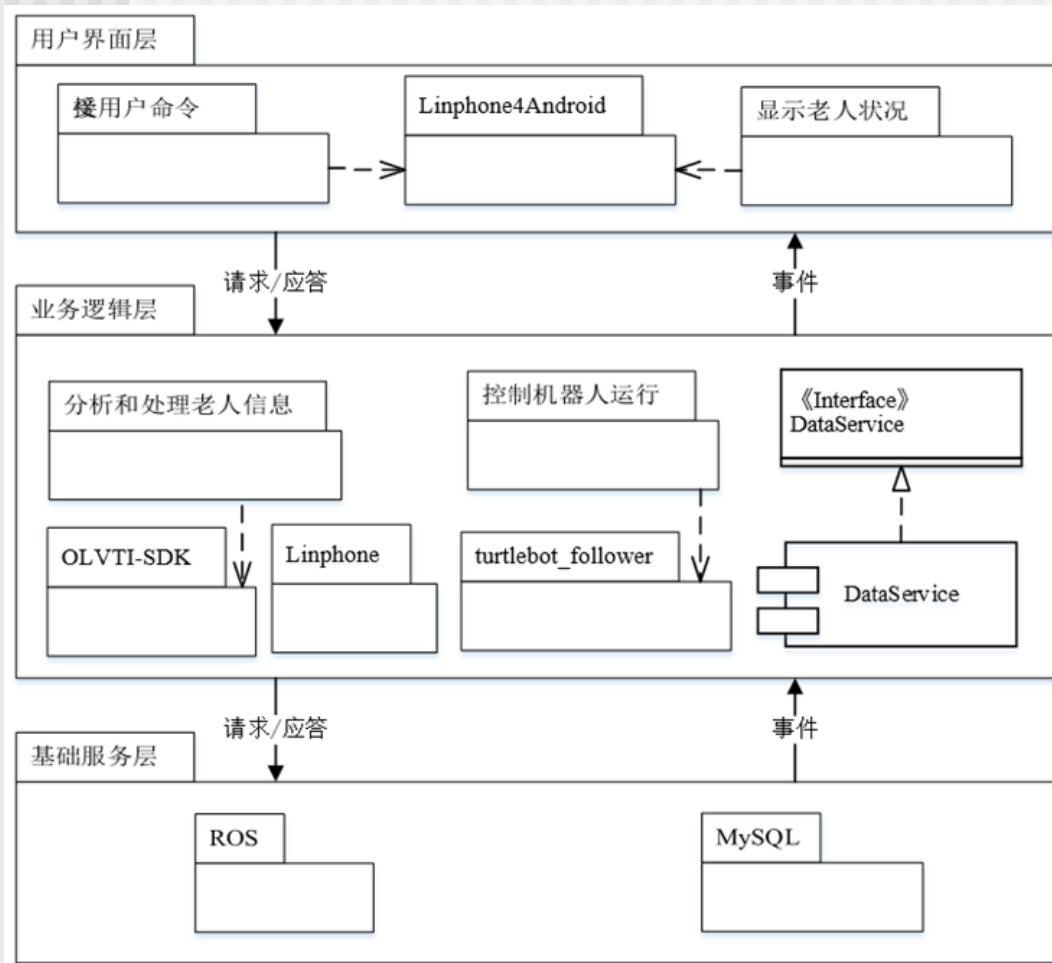
- ❑ 一组具有逻辑关联的UML模型元素（例如用例、类等）、模型图（用例图、类图、交互图、状态图、活动图等），以及其他的包
- ❑ 包在模型管理过程中是配置管理的基本单元，同时也为访问控制提供基本手段



Functions of package

- ❑ 作为软件模型的组织单元
 - ✓ 将大型软件系统划分成不同包，以构建各类模型
- ❑ 作为模型管理的基本单元
 - ✓ 以包为单位分派开发任务安排计划，包是天然的基本处理单元
- ❑ 作为系统高层结构中的组成元素。
 - ✓ 逐级细分的包才是软件高层结构中恰当的组成元素
- ❑ 作为访问控制的基本手段
 - ✓ 将包视为名字空间，每个模型元素可通过在其名称之前拼接包名，构成全系统范围内的唯一限定名来指称它，正如在文件系统中以文件的全路径名来唯一地指称一个文件

Example: Package Diagram



Component Diagram

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

Component Diagram

❑ 功效

- ✓ 描述软件系统中构件及构件间的构成和依赖关系

❑ 图的构成

- ✓ 节点：构件，具有对外接口、可分离和独立功能物理模块
- ✓ 边：构件间的依赖关系

Components and their interfaces

❑ 构件

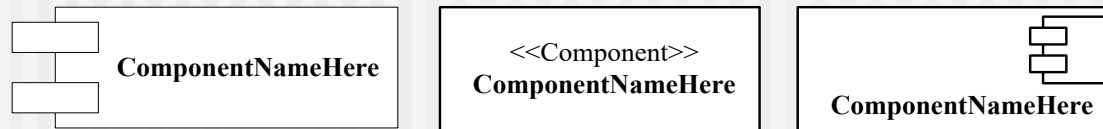
- ✓ 一个或者多个可独立部署的**执行码文件**
- ✓ 具有精确定义的**供给接口**和**需求接口**
- ✓ **可分离**，接口和实现分离
- ✓ **可替换**，构件实例可被任何实现相同接口的同一构件实例所替换

❑ 接口

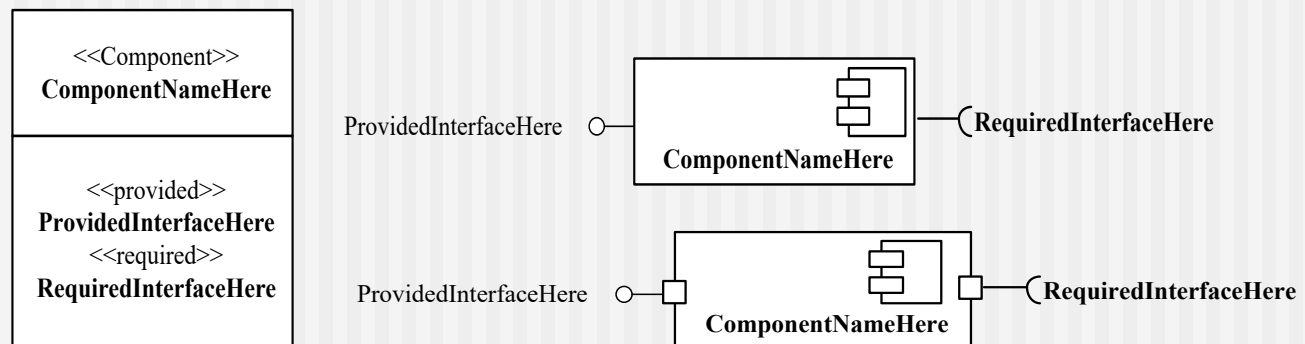
- ✓ 一组操作 和/或 属性的说明（不含操作的实现），它用作服务提供方和使用方之间的协议
- ✓ 每个构件还可以定义一些端口（port），每个端口绑定了一组**供给接口**和/或**需求接口**
- ✓ 接口由类或构件实现

Representation of components and interfaces

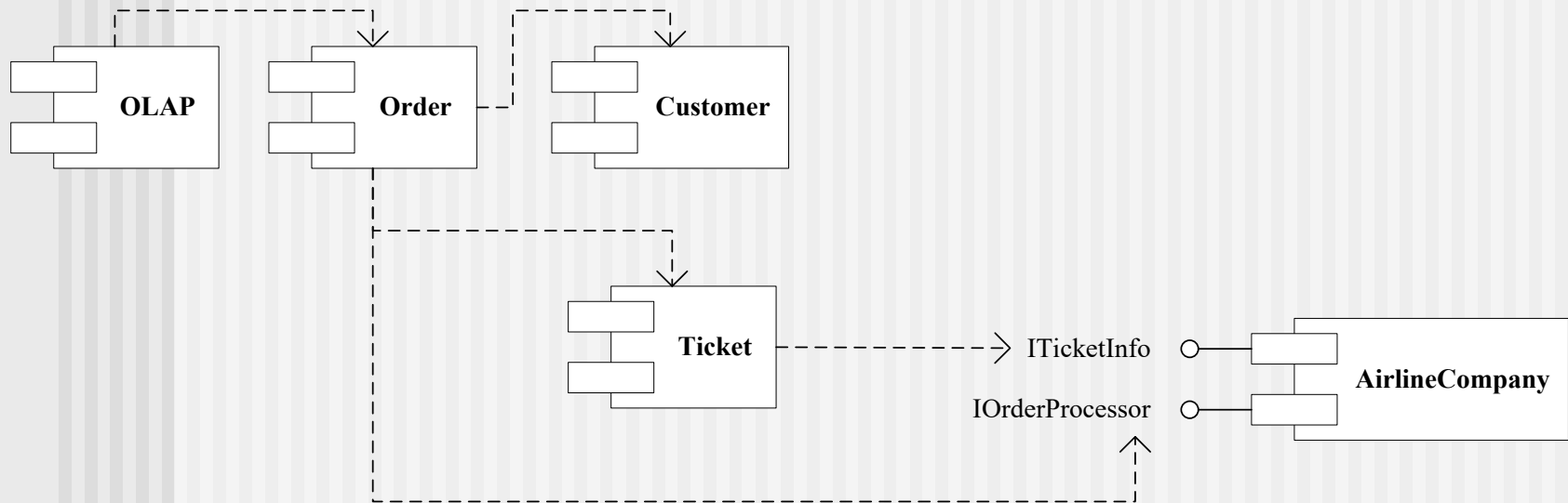
□ 构件的三种图元



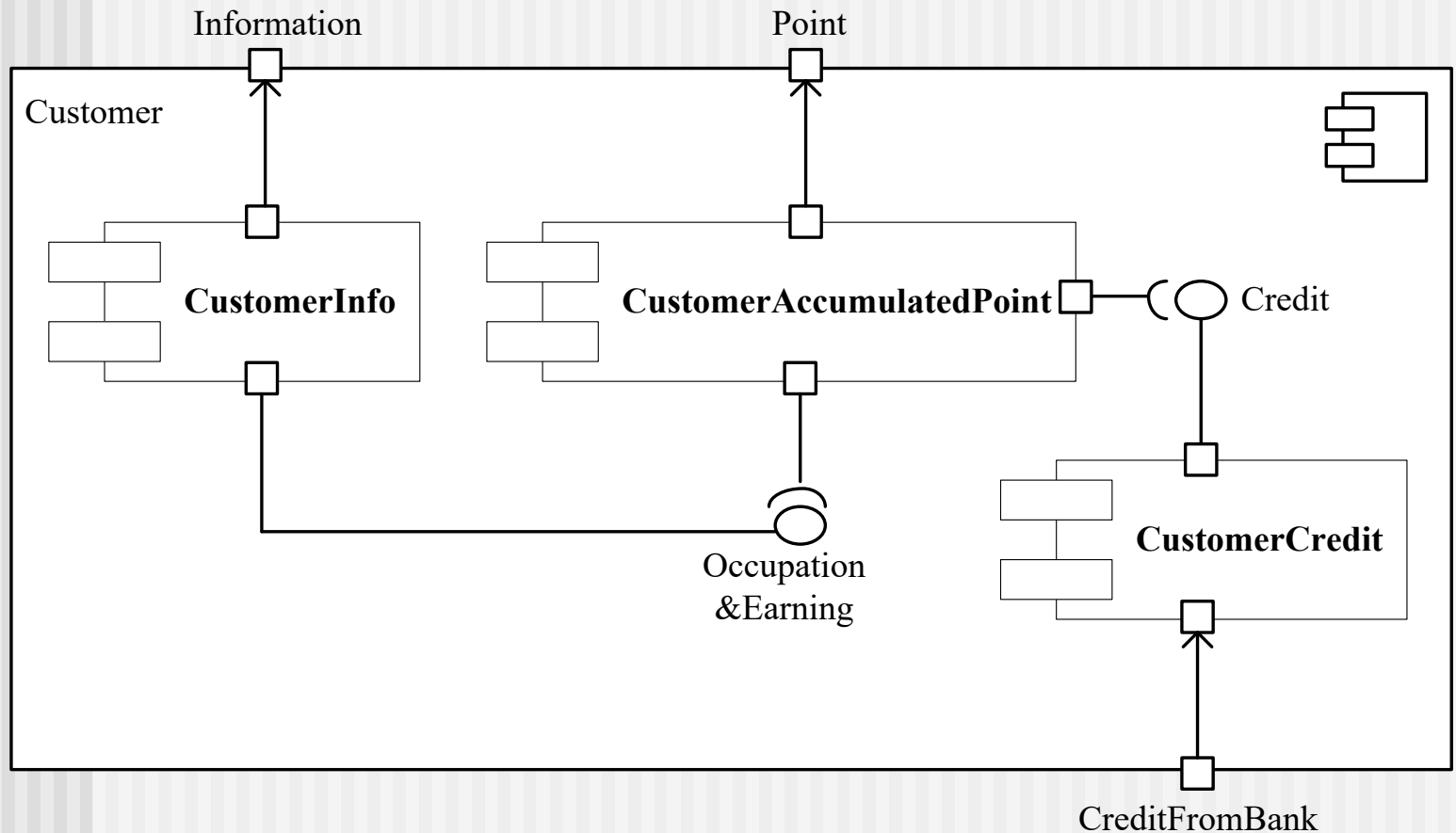
□ 构件及其接口的两种表示方法



Example: Component Diagram



Example: Component Diagram



Deployment diagram

视点	图 (diagram)	说明
结构	包图 (package diagram)	从包层面描述系统的静态结构
	类图 (class diagram)	从类层面描述系统的静态结构
	对象图 (object diagram)	从对象层面描述系统的静态结构
	构件图(component diagram)	描述系统中构件及其依赖关系
行为	状态图(statechart diagram)	描述状态的变迁
	活动图(activity diagram)	描述系统活动的实施
	通信图(communication diagram)	描述对象间的消息传递与协作
	顺序图(sequence diagram)	描述对象间的消息传递与协作
部署	部署图 (deployment diagram)	描述系统中工件在物理运行环境中的部署情况
用例	用例图 (use case diagram)	从外部用户角度描述系统功能

Deployment diagram

❑ 功效

- ✓表示软件系统可执行工件(artifact)在运行环境中的部署和分布情况
- ✓**工件**是指软件中相对独立的物理实现单元，如动态链接库 (DLL) 文件、Java类库 (jar) 文件

❑ 图的构成

- ✓节点：计算节点、工件、构件
- ✓边：通信和依赖

❑ 两种部署图

- ✓逻辑层面的**描述性部署图**描述软件的逻辑布局
- ✓物理层面的**实例性部署图**针对具体运行环境和特定的系统配置描述软件系统的物理部署情况

Nodes of descriptive deployment diagram

❑ 节点

- ✓表示软件运行环境中的一组**计算资源**（如Web服务器、应用服务器、数据库服务器等），软件工件驻留于其中，并依赖其提供的基础服务

❑ 工件

- ✓以构造型<<artifact>>标识，工件位于节点图元之内

❑ 构件

- ✓位于节点之内的构件表示该构件部署于相应节点，位于节点之外的构件主要用来指明工件与构件之间的依赖关系

❑ 与用例有关的执行者

- ✓在部署图中引入执行者意在强调执行者与部署于节点中的工件之间的使用关系及信息交互

Edge of descriptive deployment diagram

- ❑ 节点之间的通信关联

- ✓表示两个节点间的通信连接

- ❑ 工件之间的依赖关系

- ✓如果工件定义了对外接口，那么应确保工件间的依赖关系表现为工件接口上的依赖关系，如此可降低工件之间的耦合度

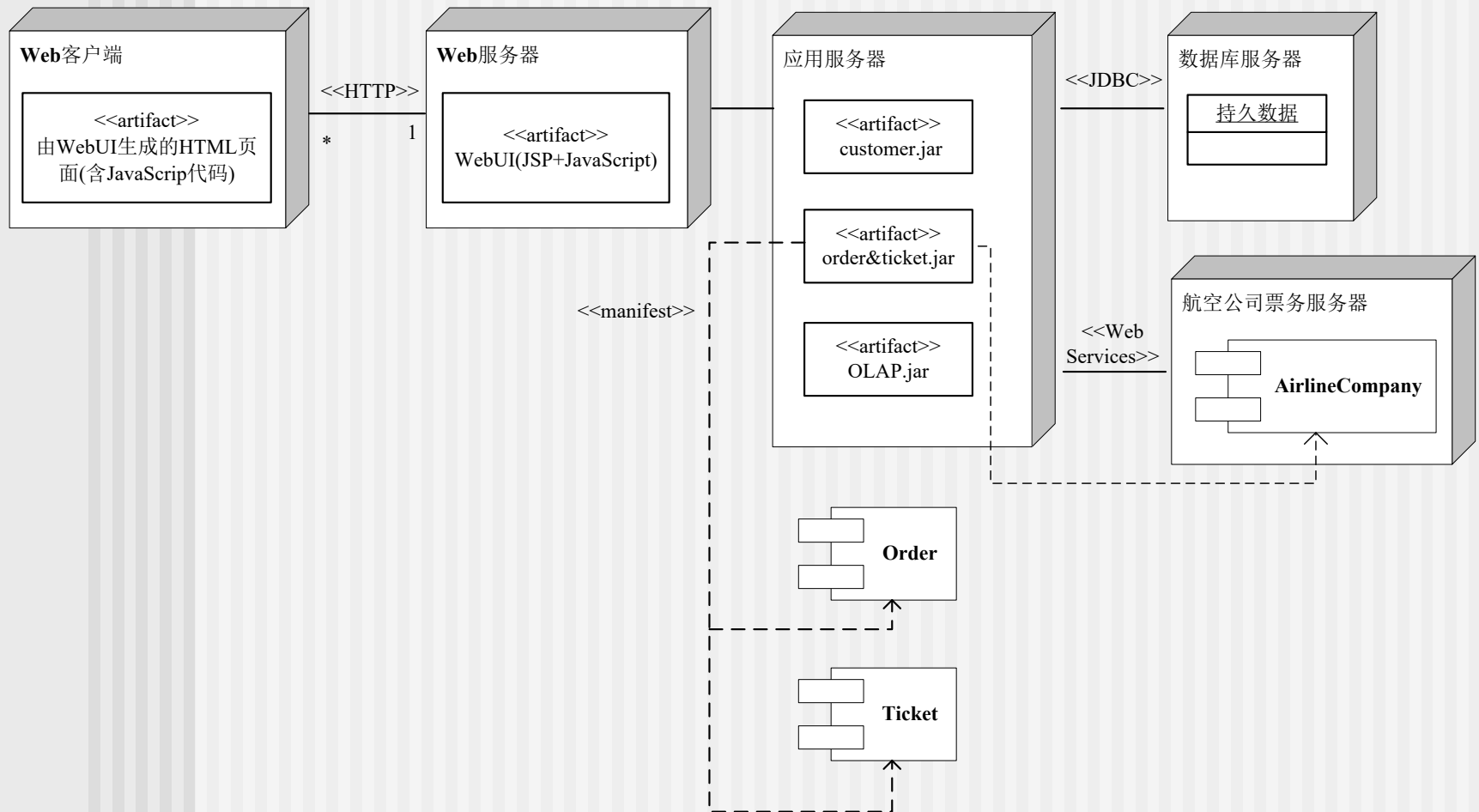
- ❑ 工件与构件之间的依赖关系

- ✓表示工件具体实现了构件

- ❑ 用例执行者与工件之间的依赖关系

- ✓表示执行者使用工件提供的服务或工件使用执行者提供的服务

Example: descriptive deployment diagram



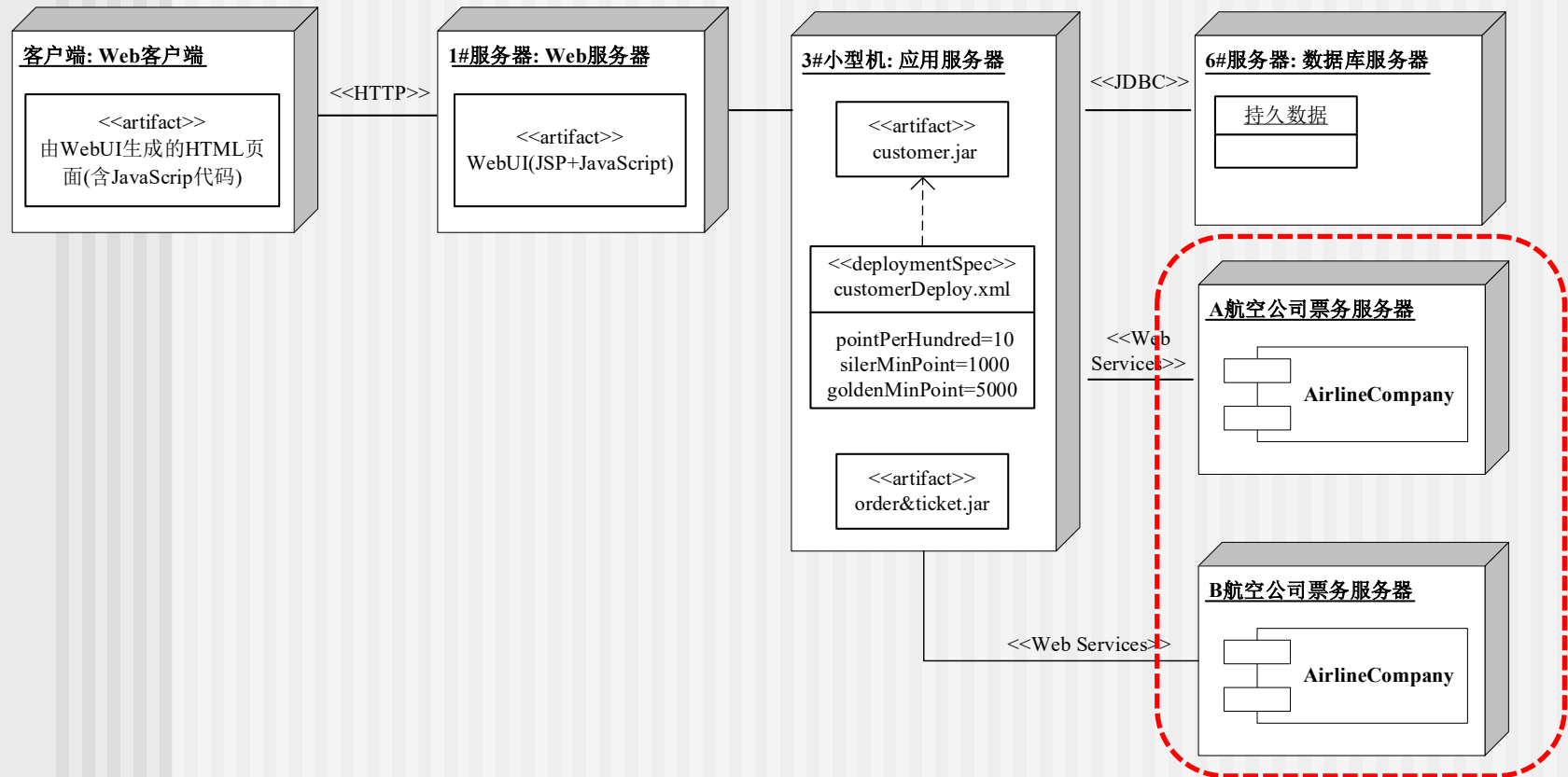
Instance deployment diagram

- ❑ 实例性部署图与描述性部署图之间的关系可类比为对象图与类图之间的关系
- ❑ 实例性部署图中节点的命名方式为“节点名: 类型名”，其中类型名为描述性部署图中的节点名

Modeling principles for deployment diagrams

- ❑ 节点按行上下对齐，按列左右对齐，边绘制成水平线段、垂直线段或者由水平、垂直两种线段组合而成的折线，避免斜线
- ❑ 工件和构件的放置应尽量使依赖关系的方向从左至右
- ❑ 供给接口位于构件的左侧，需求接口位于构件的右侧

Example: Instance deployment diagram



Architectural Genres

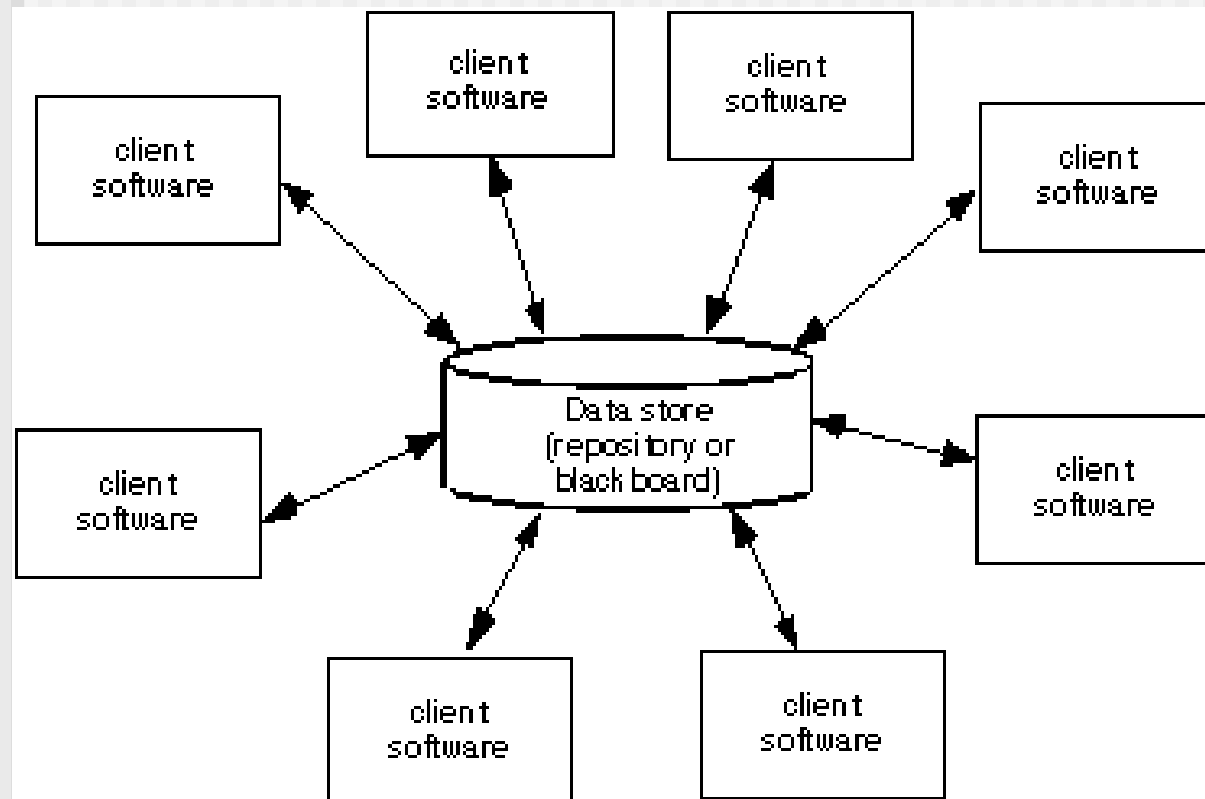
- *Genre* (类型) implies a specific category within the overall software domain.
- Within each category, you encounter a number of subcategories.
 - For example, within the genre of *buildings*, you would encounter the following general *styles*: houses, condos, apartment buildings, office buildings, industrial building, warehouses, and so on.
 - Within each general style, more specific styles might apply. Each style would have a structure that can be described using a set of predictable patterns.

Architectural Styles

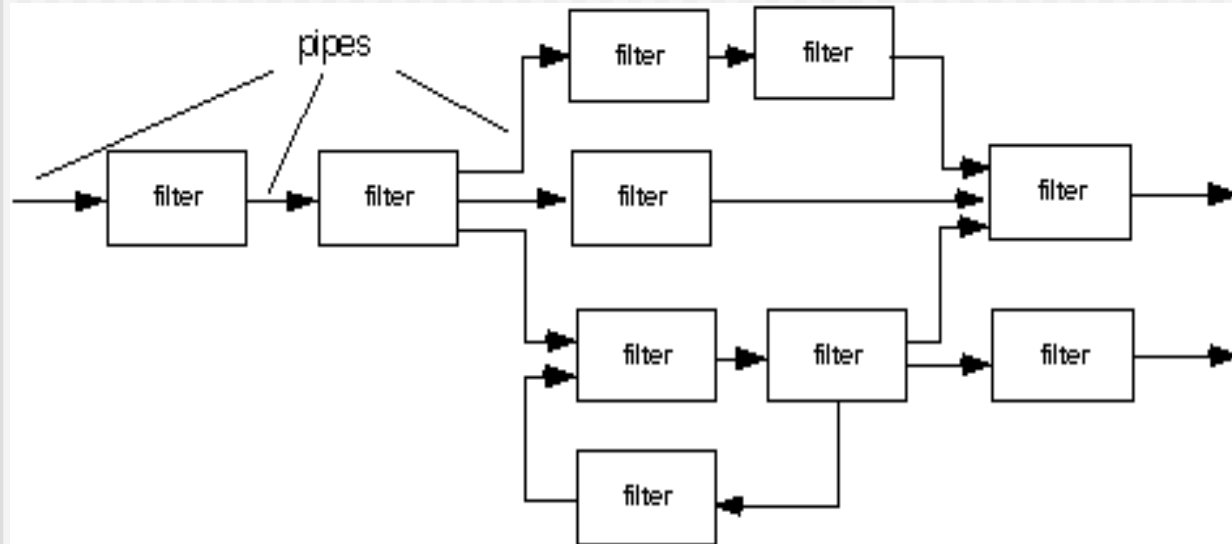
Each style describes a system category that encompasses:

- (1) a **set of components** (e.g., a database, computational modules) that perform a function required by a system,
- (2) a **set of connectors** that enable “communication, coordination and cooperation” among components,
- (3) **constraints** that define how components can be integrated to form the system.
- (4) **semantic models** that enable a designer to understand the overall properties of a system by analyzing the known properties of its constituent parts.

Data-Centered Architecture



Data Flow Architecture

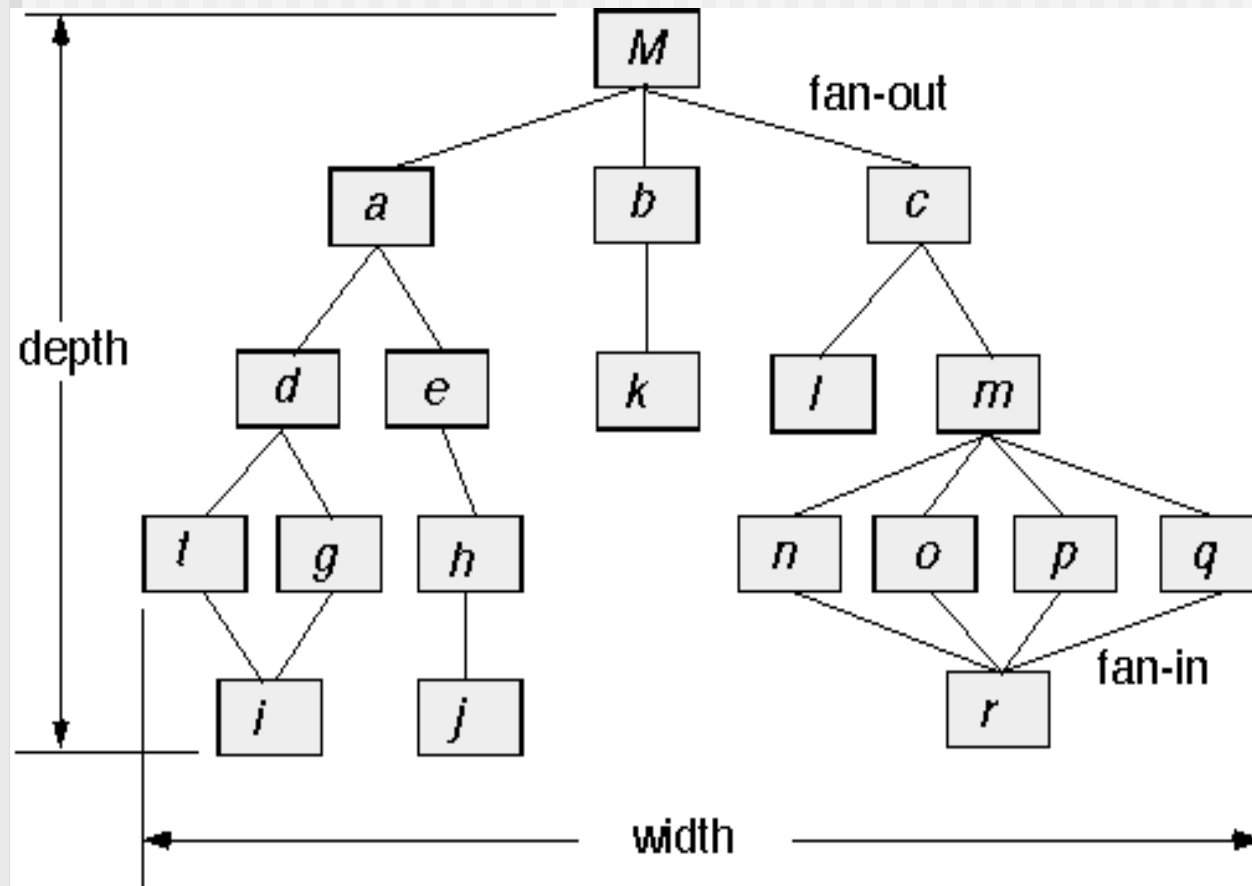


(a) pipes and filters

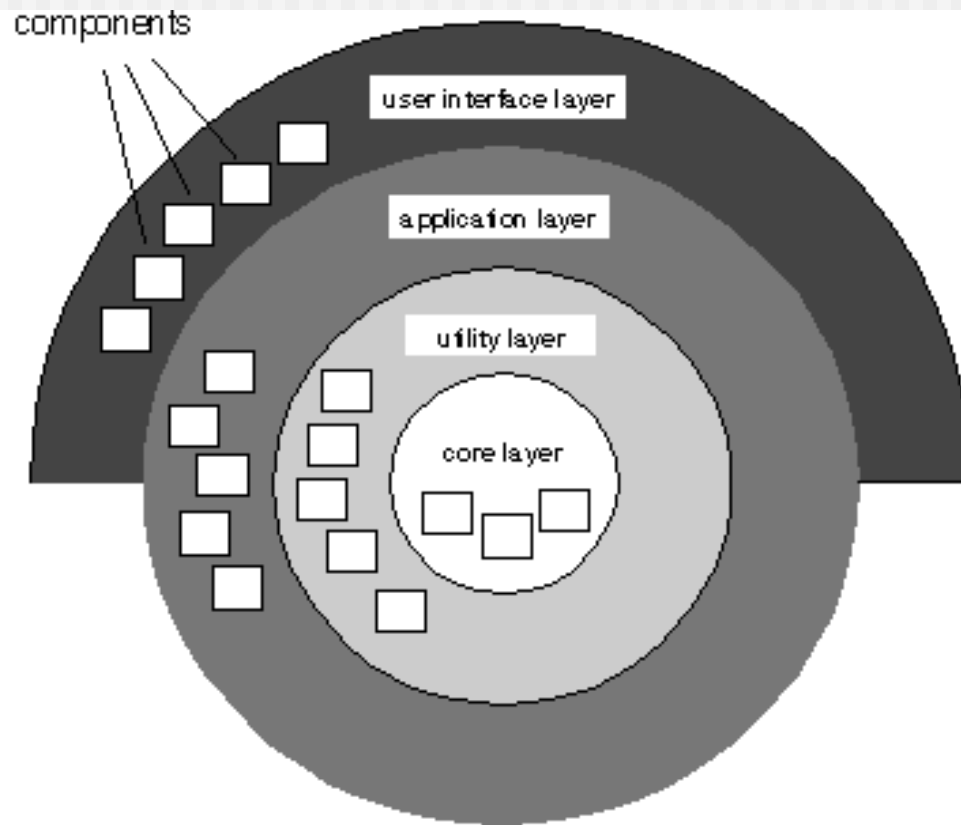


(b) batch sequential

Call and Return Architecture



Layered Architecture



Architectural Patterns

- **Concurrency**—applications must handle multiple tasks in a manner that simulates parallelism
 - *operating system process management* pattern.
 - *task scheduler* pattern.
- **Persistence**—Data persists if it survives past the execution of the process that created it. Two patterns are common:
 - a *database management system* pattern that applies the storage and retrieval capability of a DBMS to the application architecture.
 - an *application level persistence* pattern that builds persistence features into the application architecture.
- **Distribution**— the manner in which systems or components within systems communicate with one another in a distributed environment
 - A *broker* acts as a ‘middle-man’ between the client component and a server component.

Architectural Considerations

- **Economy** – The best software is uncluttered and relies on abstraction to reduce unnecessary detail.
- **Visibility** – Architectural decisions and the reasons for them should be obvious to software engineers who examine the model at a later time.
- **Spacing** – Separation of concerns in a design without introducing hidden dependencies.
- **Symmetry** (对称) – Architectural symmetry implies that a system is consistent and balanced in its attributes.
- **Emergence** (涌现) – Emergent, self-organized behavior and control.

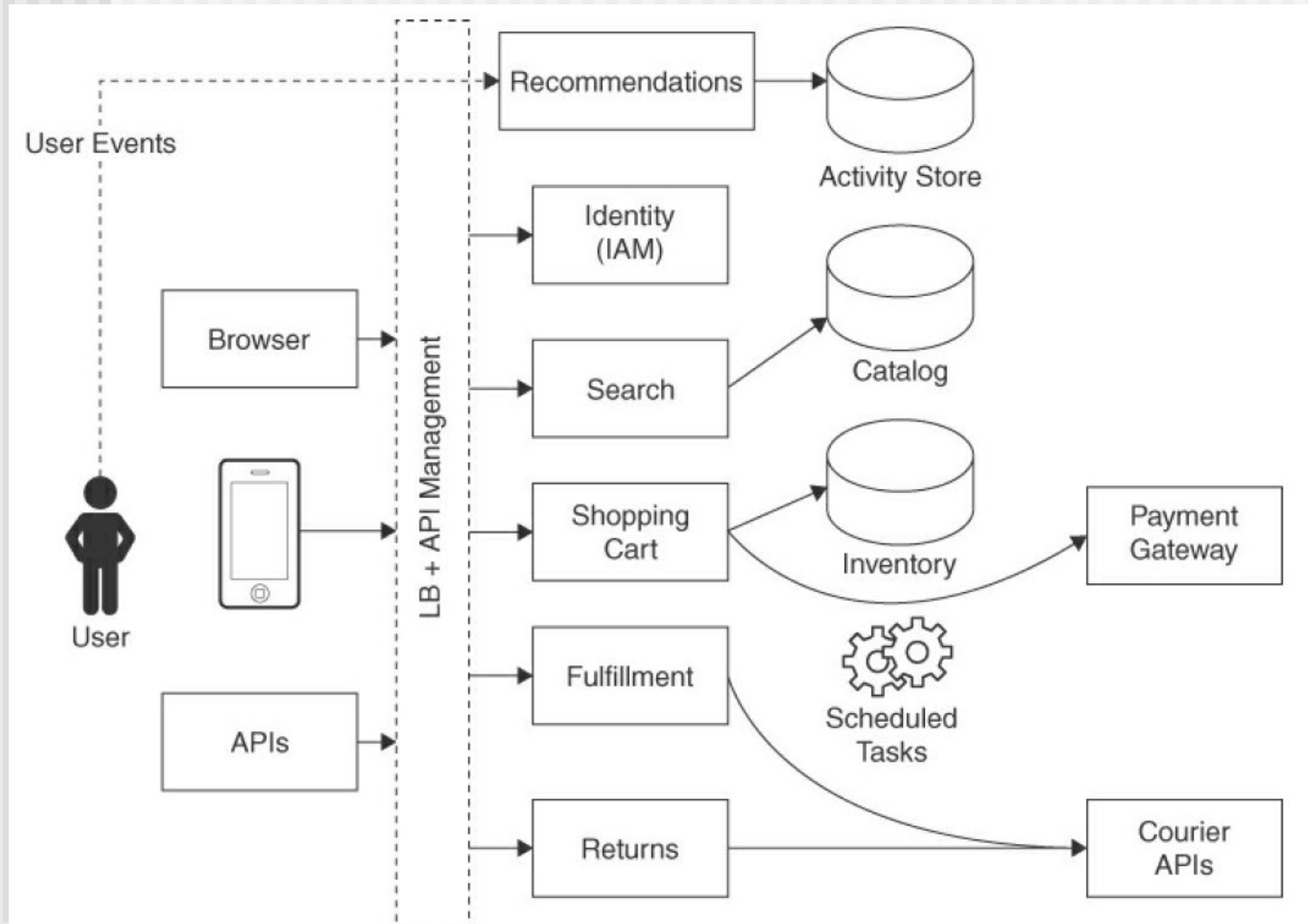
Architectural Considerations

- Five questions
 - When is the best time to market?
 - What is the skill level of the team?
 - What is our system's performance sensitivity?
 - When can we rewrite the system?
 - What are the hard problems?

Architectural Considerations

- Seven principles
 - Drive everything from the user's journey.
 - Use an iterative thin slice strategy.
 - On each iteration, add the most value for the least effort to support more users.
 - Make decisions and absorb the risks.
 - Design deeply things that are hard to change but implement them slowly.
 - Eliminate the unknowns and learn from the evidence by working on hard problems early and in parallel.
 - Understand the trade-offs between cohesion and flexibility in the software architecture.

Question? Architectural Considerations



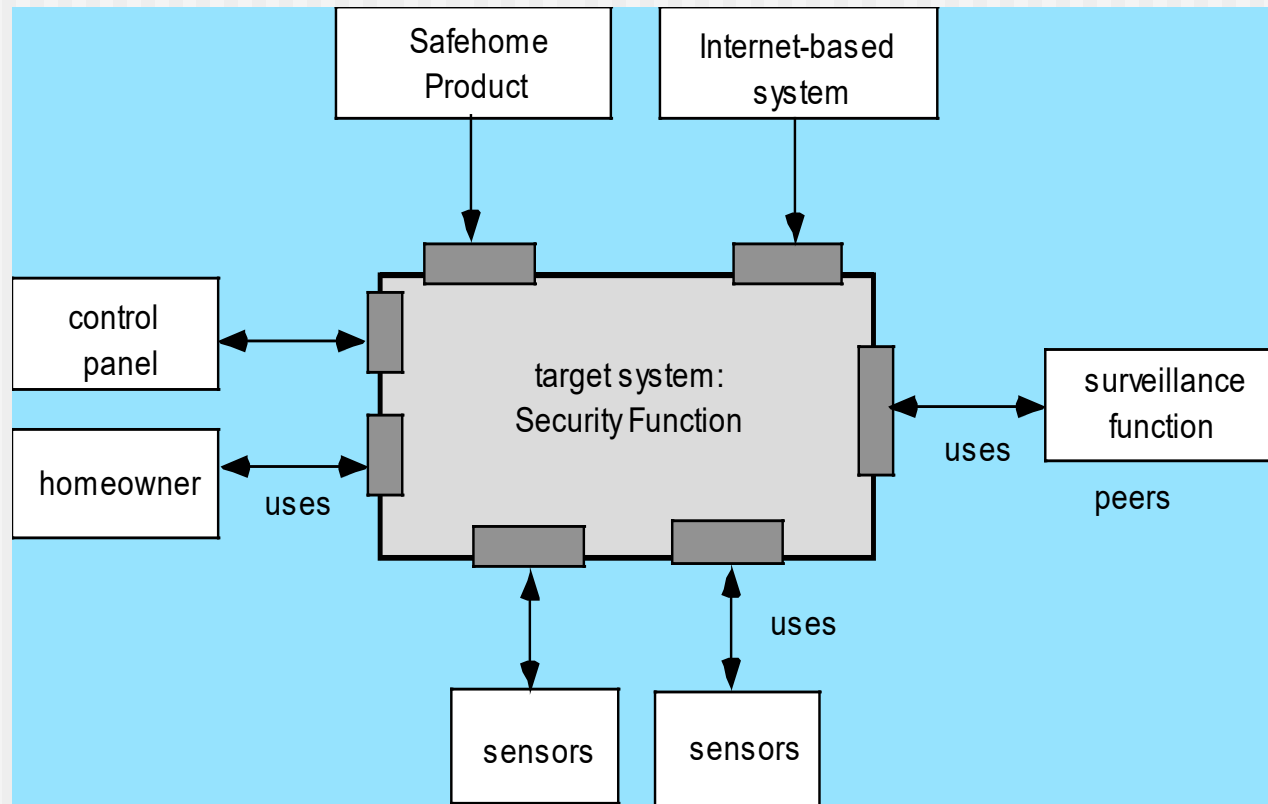
Architectural Decision Documentation

1. Determine which information items are needed for each decision.
2. Define links between each decision and appropriate requirements.
3. Provide mechanisms to change status when alternative decisions need to be evaluated.
4. Define prerequisite relationships among decisions to support traceability.
5. Link significant decisions to architectural views resulting from decisions.
6. Document and communicate all decisions as they are made.

Architectural Design

- The software must be placed into context
 - the design should define the external entities (other systems, devices, people) that the software interacts with and the nature of the interaction.
- A set of architectural archetypes should be identified
 - An *archetype* is an abstraction (similar to a class) that represents one element of system behavior.
- The designer specifies the structure of the system by defining and refining software components that implement each archetype.

Architectural Context



Archetypes

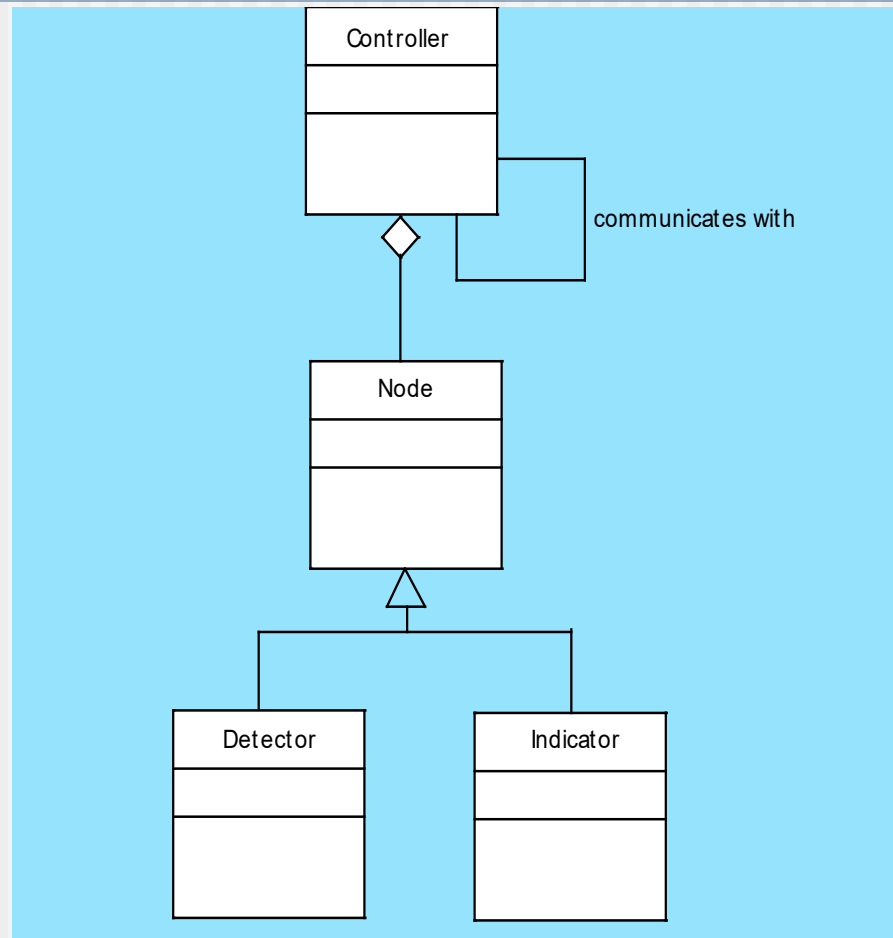
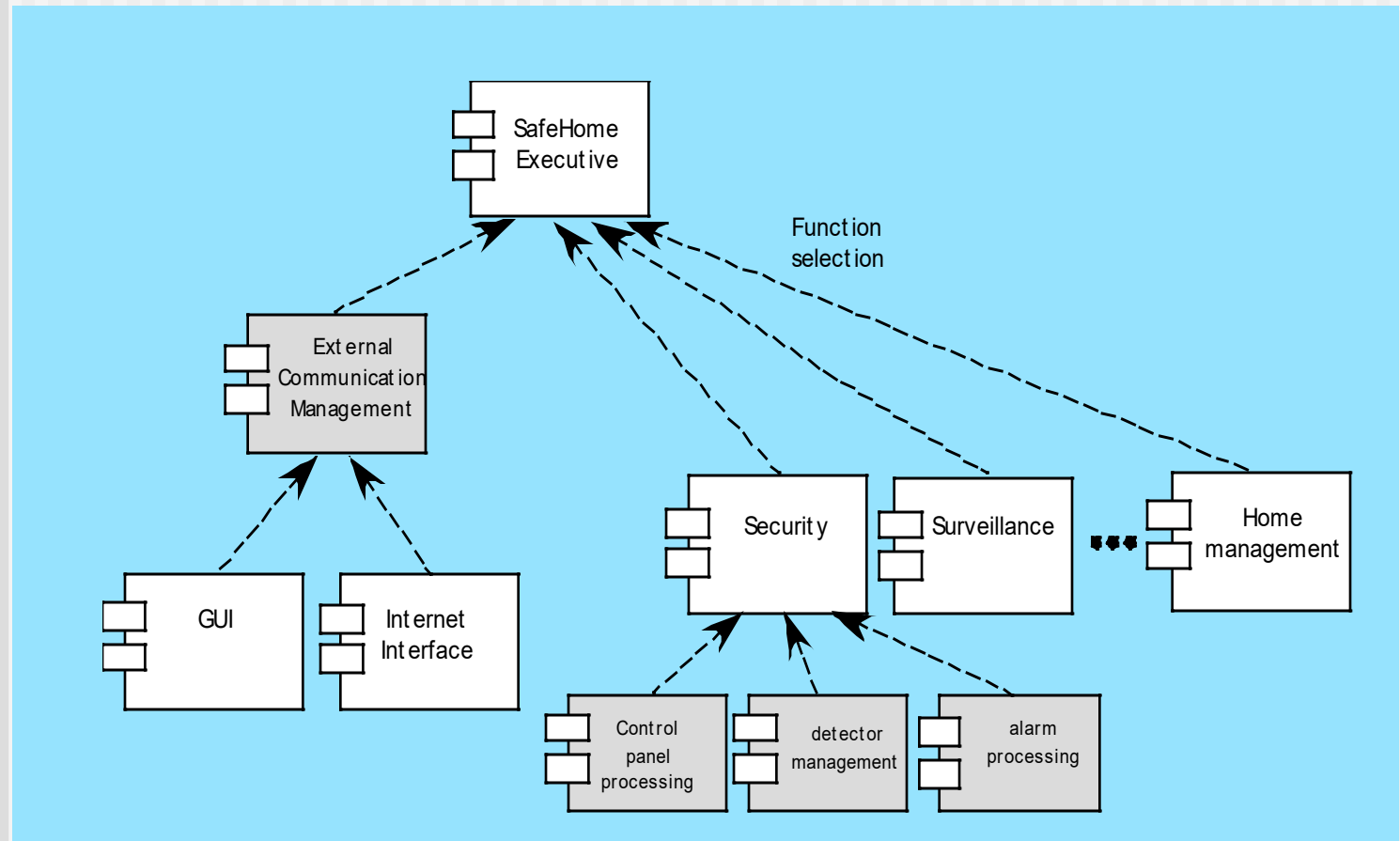
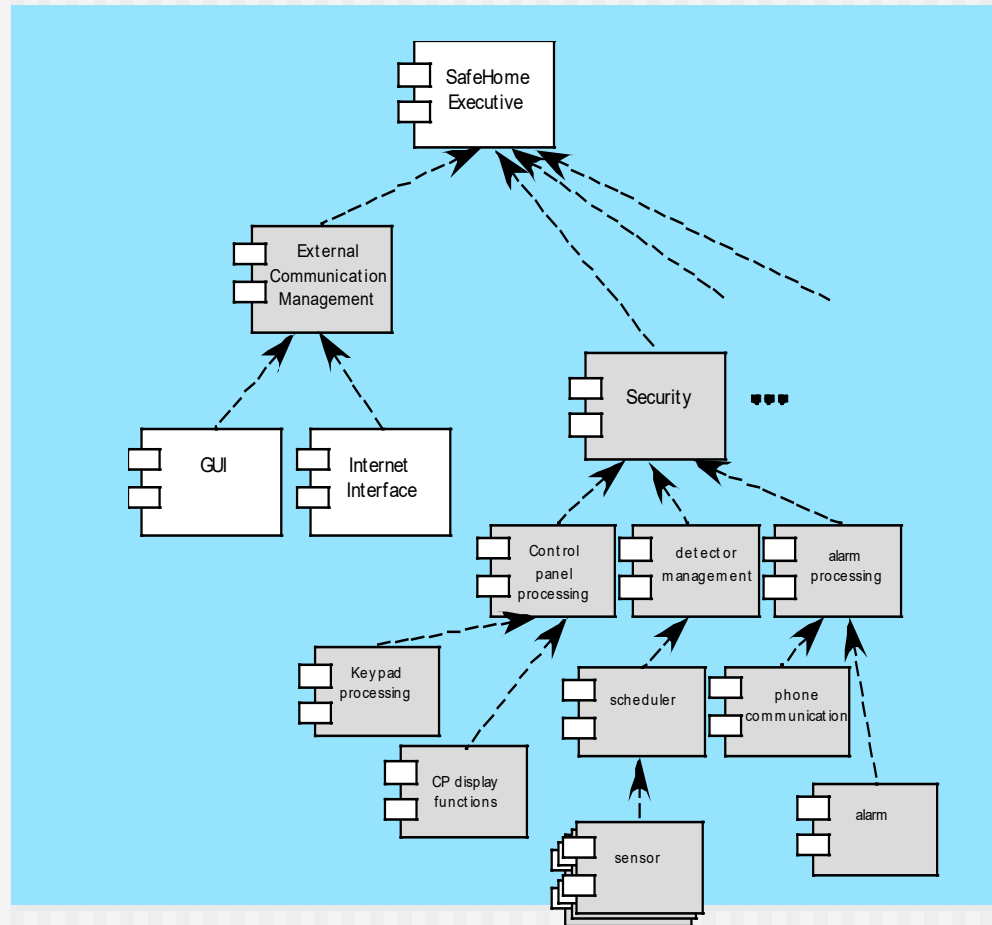


Figure 10.7 UML relationships for SafeHome security function archetypes (adapted from [BOS00])

Component Structure



Refined Component Structure



Architectural Tradeoff Analysis

1. Collect scenarios.
2. Elicit requirements, constraints, and environment description.
3. Describe the architectural styles/patterns that have been chosen to address the scenarios and requirements:
 - module view
 - process view
 - data flow view
4. Evaluate quality attributes by considered each attribute in isolation.
5. Identify the sensitivity of quality attributes to various architectural attributes for a specific architectural style.
6. Critique candidate architectures (developed in step 3) using the sensitivity analysis conducted in step 5.

Architecture Reviews

- Assess the ability of the software architecture to meet the systems quality requirements and identify potential risks.
- Have the potential to reduce project costs by detecting design problems early.
- Often make use of experience-based reviews, prototype evaluation, and scenario reviews, and checklists.

Summary

- Architectural design represents the structure of the data and program components required to build a computer-based system. A number of architectural "styles" exist. Architectural design begins with data design and proceeds to the derivation of one or more representations of the architectural structure of the system. The resulting architectural model encompasses both the data architecture and the program structure. Alternative architectural patterns are analyzed to determine the structure that is best suited to the customer's requirements. The architectural model is subjected to software quality review like all other design work products.

Assignment

■ Preview

《Software Engineering》 (8th Edition)

Chapter 14 Component-level Design

by R.S.Pressman